Can AI Detect XSS in Web Traffic?

A guide to what we built, how it works, and what it is for

Chandra Prakash Bathula

October 7, 2026

1. What this project actually is

This is a measurement experiment, not a product. The question we set out to answer is whether a machine learning detector placed in network traffic can still recognise cross-site scripting attacks after someone rewrites the payload into a different representation, and what that detector costs in response time when it runs on every request.

The important thing to understand up front is that we were never trying to build a good XSS detector. We were trying to measure one honestly. A detector that scores well is not the goal. A comparison that is fair, repeatable, and clear about its own limits is the goal. Everything described below exists to serve that.

All of it runs on the local machine. The only payload used anywhere is a harmless flag, `window.__xss_fired=true`, which sets a variable and does nothing else. No real exploit code is involved and nothing reaches the network.

2. What is running, and on which port

There are four ports in play. Three of them are servers you start yourself, and the fourth only appears while the timing benchmark is running.

Port	Process	What it does
5000	`app/vuln_app.py`	The deliberately vulnerable web application. This is the thing being attacked. It has five endpoints, and each one puts the same user input into a different place in the HTML, so the browser treats it differently. It has no user interface on purpose.
8080	`proxy/proxy.py`	The reverse proxy. Every request passes through here before reaching the app, and this is where the detector lives. It has three modes: `off` forwards without scoring, `observe` scores and logs but always forwards, and `block` returns 403 when the score crosses the threshold.
5001	`teach/playground.py`	The only part with a real interface. It has two pages. The playground lets you type an input and watch the detector score it and the browser run it side by side. The results page shows every measurement table and the figure.
8090	`bench/loadtest.py`	A temporary proxy that the benchmark starts and stops by itself, once per run. It uses a separate port so the benchmark never disturbs whatever is on 8080.

The five endpoints on port 5000 are the heart of the experiment. `/reflect-text` escapes the input before printing it, so it is safe. `/reflect-html` drops it straight into the page body. `/reflect-attr`

puts it inside an HTML attribute. `/reflect-js` puts it inside a JavaScript string. `/reflect-decode` decodes the input a second time before printing it, which lets us study what happens when the proxy and the application disagree about how many times to decode something.

The same input sent to `/reflect-text` and to `/reflect-html` produces an identical HTTP request. The detector therefore gives it an identical score. One of them is harmless and the other one runs code. That gap is the entire subject of the project.

3. What we built and why

The vulnerable app came first, and it is kept as plain as possible. If we gave it a nice interface we would be measuring the interface instead of the detector. It is a test fixture and it should stay boring.

The proxy came second. It matters that the detector sits here, because this is the only place a network defence can realistically live. It can read the request line, the headers, and the query string. It cannot see what the application will do with the value, and it cannot see whether the browser ends up running anything. We log both the raw query string and the decoded value on every request so that failures can be explained afterwards.

The dataset came third, and it took the longest. We collected 1000 labelled inputs, half attack-like and half harmless, from four sources: a Kaggle XSS dataset, the PayloadBox payload list as mirrored in SecLists, the benign rows of a large Mendeley dataset, and 50 tricky harmless strings we wrote by hand. The hand-written ones matter because they contain things like `<b>bold</b>` and `if (a < b && c > d)`, which look like attacks to a naive detector but are perfectly ordinary text.

The detector itself is deliberately simple. It breaks text into character sequences of length two to five, weights them, and runs logistic regression. The baseline model ends up with 13,617 of these features. The decision threshold, 0.617, was chosen on a validation set and then frozen. We never changed the model or the features afterwards, because the experiment compares data, not algorithms.

4. How we did it, step by step

We built the app and the proxy first and proved that a request through the proxy produced byte-for-byte the same page as a request straight to the app. Without that, nothing measured later would mean anything.

Then we cleaned and split the data. Public payload lists are full of near-copies that differ by one digit, so we grouped inputs by a skeleton key that replaces numbers and quoted strings with placeholders. Every variant we later generated inherits its parent's group and its parent's split. The code asserts that no group ever appears in two splits. If that assertion ever fires, the results are worthless, so it runs every time.

Next we trained the baseline and recorded how it does on ordinary inputs. It does very well, with an F1 of 0.995. That number is supposed to be high. It is the number the rest of the experiment knocks down.

Then we wrote the three transformation families. The first re-encodes the payload for transport using percent encoding. The second rewrites characters as HTML entities. The third rewrites the JavaScript using bracket notation, string concatenation, and hex escapes. This produced 5012 variants in total, including 1500 harmless ones used only to check the false positive rate.

Every attack variant was then opened in a real Chromium browser at its intended endpoint, and we recorded whether the marker actually fired. This gave us two separate labels that we are careful never to mix. One says the input looks like an attack. The other says the browser really ran it. A great many inputs satisfy the first and fail the second.

We also found that 56 percent of the payloads only fire when a user clicks or hovers. We record that fact rather than firing the events ourselves, because forcing a click would quietly change the meaning of the measurement.

Finally we retrained the same model with extra training data, holding the JavaScript family out of both training and validation, and measured the cost of running the detector inline under two traffic loads, three times each, with the configuration order alternated between rounds.

5. What we found

On ordinary inputs the detector catches 95 percent of attacks. On the same attacks after transformation it catches 60 percent. The confidence intervals do not overlap, so the drop is real and not an artefact of which rows landed in the test set.

The sharper result comes from comparing the two labels. Of the variants that genuinely executed in the browser, the detector flagged almost none, 0 percent for the entity family and 5 percent for the JavaScript family. Of the attack-shaped inputs that did nothing at all, it flagged between 79 and 98 percent. In other words it reacts to how text looks rather than to what it does.

Augmenting the training data appeared to fix everything, with a perfect score in every condition. We did not believe it, so we renamed the marker identifiers in the test inputs while leaving the structure and the obfuscation untouched. The score on the held-out family fell from 1.000 to 0.726, which is level with the baseline's 0.709. The improvement was mostly the model learning our marker's name. Its false positive rate also rose from 0.010 to 0.023. This is a negative result, and the brief says plainly that a negative result is valuable when it explains itself.

The cost of running the detector is small. It adds about half a millisecond to the median response time. Throughput settles at roughly 711 requests per second with the detector on and also with it off, which means the limit is the application behind it rather than the model.

6. What all of this is useful for

The direct use is that the project answers its own research question with evidence instead of opinion. It shows why the detector misses transformed inputs, which is that it learned surface strings and not behaviour. The strongest features it learned are literally the letters in the word `alert` followed by an opening bracket.

The wider use is the conclusion it supports. A detector reading HTTP requests cannot tell whether the value it is looking at will be escaped safely or dropped into a script tag, because that decision happens later and somewhere else. Our paired tests make this concrete. Eight inputs, each sent to a safe endpoint and an unsafe one, produced identical scores and opposite outcomes every time. The safe endpoint was safe because it escaped its output, not because anything was detected.

That is the practical lesson worth carrying out of this work. Detection of this kind is worth having as a signal and as a source of logs, and it is cheap enough to run inline. It is not a replacement for encoding output correctly in the application. Anyone who treats a request-content classifier as the defence has misunderstood where the vulnerability lives.

The playground on port 5001 exists to make that argument in about thirty seconds. Load the entity-encoded example, watch the detector return a verdict of ALLOW, and watch the browser run the marker anyway in the panel directly underneath.